

Week 12 skill

To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance

```
divya@Divyasree:~$ mkdir -p ~/pipeline_performance
cd ~/pipeline_performance
divya@Divyasree:~/pipeline_performance$ pwd
/home/divya/pipeline_performance
divya@Divyasree:~/pipeline_performance$ nano pipeline_monitor.c
divya@Divyasree:~/pipeline_performance$ gcc -Wall -Wextra -O2 pipeline_monitor.c -o pipeline_monitor
divya@Divyasree:~/pipeline_performance$ ls
pipeline_monitor  pipeline_monitor.c
divya@Divyasree:~/pipeline_performance$ ./pipeline_monitor
=====
PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 started. PID = 523
Job 1 completed successfully.
Job 2 started. PID = 524
Job 2 completed successfully.
Job 3 started. PID = 525
Job 3 completed successfully.
Job 4 started. PID = 526
Job 4 FAILED.

=====
PERFORMANCE ANALYSIS
=====

Job 1
Execution Time : 0.0155 seconds
CPU Time      : 16.68 ms
Status        : SUCCESS

Status        : SUCCESS

Job 3
Execution Time : 0.0154 seconds
CPU Time      : 45.34 ms
Status        : SUCCESS

Job 4
Execution Time : 0.0111 seconds
CPU Time      : 57.34 ms
Status        : FAILURE

=====
STABILITY
=====
Total Jobs      : 4
Successful Jobs : 3
Failed Jobs     : 1
Stability       : 75.00%

=====
RESOURCE MONITORING
=====
User CPU Time   : 0.057339 seconds
System CPU Time : 0.000000 seconds
Maximum RSS     : 1076 KB

=====
OVERALL PERFORMANCE
=====
Total Execution Time : 0.0531 seconds
Average Job Time     : 0.0133 seconds
Pipeline Status      : FAILURES DETECTED

Performance analysis completed.
divya@Divyasree:~/pipeline_performance$ ./pipeline_monitor > performance_report.txt
divya@Divyasree:~/pipeline_performance$ cat performance_report.txt
=====
PIPELINE PERFORMANCE MONITOR
=====
```

```
Launching 4 jobs...

Job 1 started. PID = 528
=====
PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 completed successfully.
Job 2 started. PID = 529
=====
PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 completed successfully.
Job 2 completed successfully.
Job 3 started. PID = 530
=====
PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 completed successfully.
Job 2 completed successfully.
Job 3 completed successfully.
Job 4 started. PID = 531
=====
PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 completed successfully.
Job 2 completed successfully.
Job 3 completed successfully.
```

```
=====
PERFORMANCE ANALYSIS
=====

Job 1
Execution Time : 0.0152 seconds
CPU Time      : 10.83 ms
Status        : SUCCESS

Job 2
Execution Time : 0.0118 seconds
CPU Time      : 23.49 ms
Status        : SUCCESS

Job 3
Execution Time : 0.0140 seconds
CPU Time      : 38.61 ms
Status        : SUCCESS

Job 4
Execution Time : 0.0081 seconds
CPU Time      : 47.34 ms
Status        : FAILURE

=====
STABILITY
=====
Total Jobs      : 4
Successful Jobs : 3
Failed Jobs     : 1
Stability       : 75.00%

=====
RESOURCE MONITORING
=====
User CPU Time   : 0.047335 seconds
System CPU Time : 0.000000 seconds
Maximum RSS     : 1076 KB
=====
```

```

Average Job Time      : 0.0123 seconds
Pipeline Status       : FAILURES DETECTED

Performance analysis completed.
divya@Divyasree:~/pipeline_performance$ for i in 1 2 3 4 5
do
    echo "===== RUN $i ====="
    ./pipeline_monitor
done
===== RUN 1 =====
=====
PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 started. PID = 538
Job 1 completed successfully.
Job 2 started. PID = 539
Job 2 completed successfully.
Job 3 started. PID = 540
Job 3 completed successfully.
Job 4 started. PID = 541
Job 4 FAILED.

=====
PERFORMANCE ANALYSIS
=====

Job 1
Execution Time : 0.0078 seconds
CPU Time       : 7.69 ms
Status         : SUCCESS

Job 2
Execution Time : 0.0117 seconds
CPU Time       : 19.34 ms
Status         : SUCCESS

Job 3

=====
PERFORMANCE ANALYSIS
=====

Job 1
Execution Time : 0.0068 seconds
CPU Time       : 6.65 ms
Status         : SUCCESS

Job 2
Execution Time : 0.0127 seconds
CPU Time       : 17.01 ms
Status         : SUCCESS

Job 3
Execution Time : 0.0162 seconds
CPU Time       : 33.11 ms
Status         : SUCCESS

Job 4
Execution Time : 0.0088 seconds
CPU Time       : 41.88 ms
Status         : FAILURE

=====
STABILITY
=====
Total Jobs      : 4
Successful Jobs : 3
Failed Jobs     : 1
Stability       : 75.00%

=====
RESOURCE MONITORING
=====
User CPU Time   : 0.041877 seconds
System CPU Time : 0.000000 seconds
Maximum RSS     : 1076 KB

```

```

=====
OVERALL PERFORMANCE
=====
Total Execution Time : 0.0480 seconds
Average Job Time    : 0.0120 seconds
Pipeline Status     : FAILURES DETECTED

Performance analysis completed.
divya@Divyasree:~/pipeline_performance$ nano findings.txt
divya@Divyasree:~/pipeline_performance$ ls -l
total 36
-rw-r--r-- 1 divya divya 1050 Sep 28 05:24 findings.txt
-rw-r--r-- 1 divya divya 2386 Sep 28 05:24 performance_report.txt
-rwxr-xr-x 1 divya divya 16448 Sep 28 05:23 pipeline_monitor
-rw-r--r-- 1 divya divya 5127 Sep 28 05:23 pipeline_monitor.c
divya@Divyasree:~/pipeline_performance$ gcc -Wall -Wextra -O2 pipeline_monitor.c -o pipeline_monitor
divya@Divyasree:~/pipeline_performance$ ./pipeline_monitor
=====
PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 started. PID = 570
Job 1 completed successfully.
Job 2 started. PID = 571
Job 2 completed successfully.
Job 3 started. PID = 572
Job 3 completed successfully.
Job 4 started. PID = 573
Job 4 FAILED.

=====
PERFORMANCE ANALYSIS
=====

Job 1
Execution Time : 0.0086 seconds
CPU Time      : 9.24 ms

Job 2
Execution Time : 0.0118 seconds
CPU Time      : 23.49 ms
Status        : SUCCESS

Job 3
Execution Time : 0.0140 seconds
CPU Time      : 38.61 ms
Status        : SUCCESS

Job 4
Execution Time : 0.0081 seconds
CPU Time      : 47.34 ms
Status        : FAILURE

=====
STABILITY
=====

Total Jobs      : 4
Successful Jobs : 3
Failed Jobs     : 1
Stability       : 75.00%

=====
RESOURCE MONITORING
=====

User CPU Time   : 0.047335 seconds
System CPU Time : 0.000000 seconds
Maximum RSS     : 1076 KB

=====
OVERALL PERFORMANCE
=====

Total Execution Time : 0.0491 seconds
Average Job Time     : 0.0123 seconds
Pipeline Status      : FAILURES DETECTED

Performance analysis completed.
divya@Divyasree:~/pipeline_performance$ |

```

Code:

```
GNU nano 7.2 pip
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/resource.h>
#include <time.h>

#define JOBS 4

typedef struct
{
    int id;
    int status;
    double execution_time;
    long cpu_time;
} Job;

void run_job(int job_id)
{
    /*
     * Simulate different workloads.
     * Job 4 intentionally fails so that
     * failure detection can be demonstrated.
     */

    if (job_id == 1)
    {
        for (volatile long i = 0; i < 300000000; i++)
            ;
        exit(0);
    }

    if (job_id == 2)
    {
        for (volatile long i = 0; i < 500000000; i++)
            ;
        exit(0);
    }

    if (job_id == 3)
    {
        for (volatile long i = 0; i < 700000000; i++)
            ;
        exit(0);
    }
}
```

```

}

int main()
{
    Job jobs[JOBS];

    printf("=====\n");
    printf("      PIPELINE PERFORMANCE MONITOR\n");
    printf("=====\n");

    printf("\nLaunching %d jobs...\n", JOBS);

    /*
     * Launch multiple jobs
     */
    for (int i = 0; i < JOBS; i++)
    {
        jobs[i].id = i + 1;

        struct timespec start, end;

        clock_gettime(CLOCK_MONOTONIC, &start);

        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
            jobs[i].status = -1;
            continue;
        }

        if (pid == 0)
        {
            printf("Job %d started. PID = %d\n",
                i + 1, getpid());

            run_job(i + 1);
        }

        int status;

        waitpid(pid, &status, 0);

        /*
         * Performance analysis
         */

        printf("\n=====\n");
        printf("      PERFORMANCE ANALYSIS\n");
        printf("=====\n");

        double total_time = 0;
        int successful = 0;
        int failed = 0;

        for (int i = 0; i < JOBS; i++)
        {
            printf("\nJob %d\n", jobs[i].id);

            printf("Execution Time : %.4f seconds\n",
                jobs[i].execution_time);

            printf("CPU Time      : %.2f ms\n",
                jobs[i].cpu_time / 1000.0);

            if (jobs[i].status == 0)
            {
                printf("Status      : SUCCESS\n");
                successful++;
            }
            else
            {
                printf("Status      : FAILURE\n");
                failed++;
            }

            total_time += jobs[i].execution_time;
        }

        /*

```

```

GNU nano 7.2 pipeline_monitor.c
/*
 * Resource monitoring
 */

struct rusage usage;

getrusage(RUSAGE_CHILDREN, &usage);

printf("\n=====\\n");
printf("          RESOURCE MONITORING\\n");
printf("=====\\n");

printf("User CPU Time   : %ld.%06ld seconds\\n",
       usage.ru_utime.tv_sec,
       usage.ru_utime.tv_usec);

printf("System CPU Time  : %ld.%06ld seconds\\n",
       usage.ru_stime.tv_sec,
       usage.ru_stime.tv_usec);

printf("Maximum RSS     : %ld KB\\n",
       usage.ru_maxrss);

/*
 * Overall performance
 */

printf("\n=====\\n");
printf("          OVERALL PERFORMANCE\\n");
printf("=====\\n");

printf("Total Execution Time : %.4f seconds\\n",
       total_time);

printf("Average Job Time     : %.4f seconds\\n",
       total_time / JOBS);

if (failed == 0)
{
    printf("Pipeline Status      : STABLE\\n");
}
else
{
    printf("Pipeline Status      : FAILURES DETECTED\\n");
}

```

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation

```

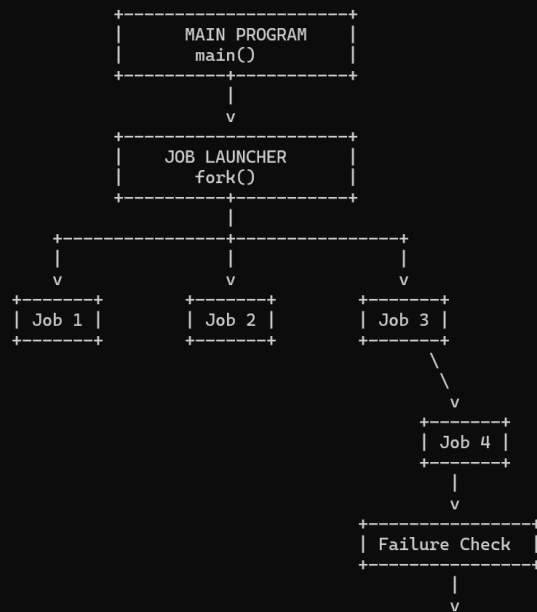
divya@Divyasree:~$ cd ~/pipeline_performance
divya@Divyasree:~/pipeline_performance$ pwd
ls
/home/divya/pipeline_performance
findings.txt      pipeline_monitor
performance_report.txt  pipeline_monitor.c
divya@Divyasree:~/pipeline_performance$ nano architecture.md
divya@Divyasree:~/pipeline_performance$ nano flow_diagram.txt
divya@Divyasree:~/pipeline_performance$ nano syscalls.txt
divya@Divyasree:~/pipeline_performance$ nano design_decisions.txt
divya@Divyasree:~/pipeline_performance$ nano documentation_review.txt
divya@Divyasree:~/pipeline_performance$ ls -l
total 60
-rw-r--r-- 1 divya divya 7575 Sep 28 05:37 architecture.md
-rw-r--r-- 1 divya divya 1559 Sep 28 05:38 design_decisions.txt
-rw-r--r-- 1 divya divya 1390 Sep 28 05:38 documentation_review.txt
-rw-r--r-- 1 divya divya 1050 Sep 28 05:24 findings.txt
-rw-r--r-- 1 divya divya 2205 Sep 28 05:37 flow_diagram.txt
-rw-r--r-- 1 divya divya 2386 Sep 28 05:24 performance_report.txt
-rwxr-xr-x 1 divya divya 16448 Sep 28 05:25 pipeline_monitor
-rw-r--r-- 1 divya divya 5127 Sep 28 05:23 pipeline_monitor.c
-rw-r--r-- 1 divya divya 1292 Sep 28 05:37 syscalls.txt
divya@Divyasree:~/pipeline_performance$ cat architecture.md
# PIPELINE PERFORMANCE MONITOR
## System Architecture Documentation

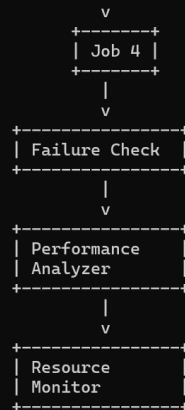
```

1. Project Overview

The Pipeline Performance Monitor is a Linux-based C program designed to execute multiple jobs, monitor their execution, detect failures, measure performance and analyze resource usage.

Architecture:





3. Main Components

Main Controller

The main() function controls the complete execution.

Responsibilities:

- Initialize job information

Main Controller

The main() function controls the complete execution.

Responsibilities:

- Initialize job information
- Launch jobs
- Wait for child processes
- Collect exit status
- Calculate performance
- Display final results

Job Launcher

The job launcher creates child processes using fork().

Each child receives a unique job ID.

The parent process waits for the child using waitpid().

Child Job Processor

The run_job() function performs the workload.

Four jobs are executed:

Job 1 - CPU workload

```
Job 1 - CPU workload
Job 2 - CPU workload
Job 3 - CPU workload
Job 4 - CPU workload followed by intentional failure
```

```
---
```

Process Monitor

The parent process monitors every child process.

It checks:

- Process completion
- Exit status
- Successful execution
- Failure conditions

```
---
```

Performance Analyzer

The performance analyzer measures:

- Execution time
- CPU time
- Average execution time
- Total execution time

```
---
```

The resource monitor uses `getrusage()`.

It collects:

- User CPU time
- System CPU time
- Maximum resident memory

```
---
```

4. Execution Flow

1. Program starts.
2. Four jobs are prepared.
3. Parent creates a child process.
4. Child executes its workload.
5. Parent waits for completion.
6. Parent checks exit status.
7. Execution time is calculated.
8. Resource information is collected.
9. Results are stored.
10. All jobs are analyzed.
11. Stability percentage is calculated.
12. Final performance information is displayed.
13. Program terminates.

```
---
```

5. Execution Paths

Successful Execution Path

```
main()
```

```

v
Child Process
|
v
run_job()
|
v
exit(0)
|
v
Parent waitpid()
|
v
Status = SUCCESS
|
v
Performance Analysis

```

Failure Execution Path

```

main()
|
v
fork()
|
v
Child Process
|
v
run_job()
exit(1)
|
v
Parent waitpid()
|
v
Status = FAILURE
|
v
Failure Detected
|
v
Performance Analysis

```

6. System Calls

fork()

Creates a new child process.

Purpose:

To execute multiple jobs independently.

waitpid()

Allows the parent process to wait for a specific child process.

exit()

Terminates the child process.

exit(0) indicates successful execution.

exit(1) indicates failure.

clock_gettime()

Obtains high-resolution timing information.

Purpose:

To measure job execution time.

getrusage()

Obtains resource usage information.

Purpose:

To monitor:

- CPU usage
- System CPU usage
- Maximum resident memory

Displays execution and performance information to the terminal.

7. Data Flow

Input:

Job configuration

|
v

Main Controller

|
v

Process Creation

|
v

Child Jobs

|
v

Execution

|
v

8. Design Decisions

Decision 1: Use fork()

fork() was selected because it is a standard Linux mechanism for creating child processes.

Decision 2: Use waitpid()

waitpid() provides controlled synchronization and allows the parent to inspect child termination.

Decision 3: Use exit status

Exit status provides a simple mechanism to identify successful and failed jobs.

Decision 4: Use clock_gettime()

CLOCK_MONOTONIC provides reliable elapsed-time measurement.

Decision 5: Use getrusage()

getrusage() provides Linux resource usage information for child processes.

Decision 6: Fixed Four Jobs

Four jobs were selected to demonstrate multiple job execution and performance comparison without making the demonstration unnecessarily complex.

9. Stability Calculation

Stability is calculated using:

$$\text{Stability} = (\text{Successful Jobs} / \text{Total Jobs}) \times 100$$

For example:

3 successful jobs
4 total jobs

$$\text{Stability} = 3 / 4 \times 100$$
$$\text{Stability} = 75\%$$

10. Error Handling

The program checks:

- fork() failure
- Child process exit status
- Job failures
- Resource usage
- Execution completion

When a child exits with a non-zero status, the job is reported as failed.

11. Performance Metrics

The following metrics are analyzed:

1. Individual execution time
2. CPU time
3. Total execution time
4. Average execution time
5. Successful jobs
6. Failed jobs
7. Stability percentage
8. Maximum resident memory

12. Conclusion

The Pipeline Performance Monitor provides a structured Linux process-management architecture.

It demonstrates process creation, synchronization, failure detection, resource monitoring and performance analysis.

The architecture separates job execution from monitoring and analysis, making the system easier to understand, test and extend.

divya@Divyasree:~/pipeline_performance\$ cat flow_diagram.txt

```
=====
                        PIPELINE EXECUTION FLOW
=====
```

PIPELINE EXECUTION FLOW



```

2. waitpid()
-----
Purpose:
Waits for a specific child process.

Used for:
Synchronizing parent and child.

It also provides the child's termination status.

```

```

3. exit()
-----
Purpose:
Terminates a process.

exit(0):
Successful execution.

exit(1):
Failure.

```

```

4. clock_gettime()
-----
Purpose:
Obtains high-resolution system time.

Used for:
Measuring job execution duration.

CLOCK_MONOTONIC:
Used because it provides elapsed time
that is not affected by normal system
clock changes.

```

```

5. getrusage()
-----
Purpose:

```

```

6. printf()
-----
Purpose:
Displays system status, job results,
performance measurements and errors.

```

SYSTEM CALL EXECUTION RELATIONSHIP

=====

```

main()
|
|-- fork()
|   |
|   |-- Child
|       |
|       |-- run_job()
|       |
|       |-- exit()
|
|-- waitpid()
|
|-- clock_gettime()
|
|-- getrusage()
|
|-- printf()
===== DESIGN DECISIONS =====
DESIGN DECISIONS
=====

```

Project:
Pipeline Performance Monitor

=====

Project:
Pipeline Performance Monitor

1. Process Model -----

The program uses a parent-child process model.

Reason:
It demonstrates Linux process management clearly.

2. Job Count -----

Four jobs are used.

Reason:
Four jobs provide enough workload to demonstrate multiple execution paths and performance comparison.

3. Process Creation -----

fork() is used.

Reason:
fork() is a standard Linux system call for creating child processes.

4. Process Synchronization -----

waitpid() is used.

Reason:
The parent must wait for each child and obtain its termination status.

6. Timing -----

clock_gettime() with CLOCK_MONOTONIC is used.

Reason:
It provides reliable elapsed-time measurement.

7. Resource Monitoring -----

getrusage() is used.

Reason:
It provides CPU and memory-related resource information.

8. Stability Measurement -----

Stability is calculated from successful and total jobs.

Formula:

Successful Jobs
----- x 100
Total Jobs

9. Reporting -----

Results are displayed on the terminal and can also be redirected to a report file.

Reason:
This makes testing and documentation easier.

```
ignored.
===== DOCUMENTATION REVIEW =====
DOCUMENTATION REVIEW
=====

Project:
Pipeline Performance Monitor

Review Checklist
-----

[YES] Architecture documented
[YES] Components identified
[YES] Execution flow documented
[YES] Successful execution path documented
[YES] Failure execution path documented
[YES] System calls documented
[YES] Data flow documented
[YES] Design decisions recorded
[YES] Performance metrics documented
[YES] Error handling documented
[YES] Stability calculation documented
[YES] Resource monitoring documented

Architecture Review
-----

The architecture clearly separates:

- Job execution
- Process monitoring
- Resource monitoring
- Performance analysis

Execution Review
-----

The parent process creates child processes,
waits for their completion and checks their
-----

The parent process creates child processes,
waits for their completion and checks their
exit status.

Failure Review
-----

A non-zero child exit status is identified
as a failure.

Performance Review
-----

Execution time and CPU/resource information
are collected and analyzed.

Documentation Quality
-----

The documentation describes the purpose,
execution flow, system calls, architecture,
design decisions and performance analysis.

Final Review Result
-----

DOCUMENTATION COMPLETE

The system architecture and execution behavior
are documented sufficiently for understanding,
testing and future modification.
divya@Divyasree:~/pipeline_performance$ |
```

To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables

```
divya@Divyasree:~$ cd ~/pipeline_performance
divya@Divyasree:~/pipeline_performance$ pwd
ls
/home/divya/pipeline_performance
architecture.md          performance_report.txt
design_decisions.txt      pipeline_monitor
documentation_review.txt pipeline_monitor.c
findings.txt            syscalls.txt
flow_diagram.txt
divya@Divyasree:~/pipeline_performance$ mkdir -p src docs reports build
divya@Divyasree:~/pipeline_performance$ mv pipeline_monitor.c src/
divya@Divyasree:~/pipeline_performance$ mv architecture.md docs/
mv flow_diagram.txt docs/
mv syscalls.txt docs/
mv design_decisions.txt docs/
mv documentation_review.txt docs/
mv findings.txt docs/
divya@Divyasree:~/pipeline_performance$ mv performance_report.txt reports/
divya@Divyasree:~/pipeline_performance$ nano .gitignore
divya@Divyasree:~/pipeline_performance$ nano Makefile
divya@Divyasree:~/pipeline_performance$ nano README.md
divya@Divyasree:~/pipeline_performance$ make
mkdir -p build
gcc -Wall -Wextra -O2 -g src/pipeline_monitor.c -o build/pipeline_monitor
divya@Divyasree:~/pipeline_performance$ make run
./build/pipeline_monitor
=====
PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 started. PID = 1377

Job 1 started. PID = 1377
Job 1 completed successfully.
Job 2 started. PID = 1378
Job 2 completed successfully.
Job 3 started. PID = 1379
Job 3 completed successfully.
Job 4 started. PID = 1380
Job 4 FAILED.

=====
PERFORMANCE ANALYSIS
=====

Job 1
Execution Time : 0.0092 seconds
CPU Time      : 6.12 ms
Status        : SUCCESS

Job 2
Execution Time : 0.0114 seconds
CPU Time      : 17.69 ms
Status        : SUCCESS

Job 3
Execution Time : 0.0181 seconds
CPU Time      : 36.05 ms
Status        : SUCCESS

Job 4
Execution Time : 0.0088 seconds
CPU Time      : 44.95 ms
Status        : FAILURE
```

```
Total Jobs      : 4
Successful Jobs : 3
Failed Jobs     : 1
Stability       : 75.00%

=====
                RESOURCE MONITORING
=====
User CPU Time   : 0.044949 seconds
System CPU Time : 0.003060 seconds
Maximum RSS     : 1032 KB

=====
                OVERALL PERFORMANCE
=====
Total Execution Time : 0.0475 seconds
Average Job Time     : 0.0119 seconds
Pipeline Status      : FAILURES DETECTED

Performance analysis completed.
divya@Divyasree:~/pipeline_performance$ ./build/pipeline_monitor
=====
                PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 started. PID = 1418
Job 1 completed successfully.
Job 2 started. PID = 1419
Job 2 completed successfully.
Job 3 started. PID = 1420
Job 3 completed successfully.

=====
                PERFORMANCE ANALYSIS
=====

Job 1
Execution Time : 0.0089 seconds
CPU Time       : 8.96 ms
Status         : SUCCESS

Job 2
Execution Time : 0.0111 seconds
CPU Time       : 20.16 ms
Status         : SUCCESS

Job 3
Execution Time : 0.0163 seconds
CPU Time       : 36.64 ms
Status         : SUCCESS

Job 4
Execution Time : 0.0133 seconds
CPU Time       : 50.01 ms
Status         : FAILURE

=====
                STABILITY
=====
Total Jobs      : 4
Successful Jobs : 3
Failed Jobs     : 1
Stability       : 75.00%

=====
```

```

=====
OVERALL PERFORMANCE
=====
Total Execution Time : 0.0496 seconds
Average Job Time    : 0.0124 seconds
Pipeline Status     : FAILURES DETECTED

Performance analysis completed.
divya@Divyasree:~/pipeline_performance$ ^C
divya@Divyasree:~/pipeline_performance$ make clean
rm -rf build
divya@Divyasree:~/pipeline_performance$ make clean
make
rm -rf build
mkdir -p build
gcc -Wall -Wextra -O2 -g src/pipeline_monitor.c -o build/pipeline_monitor
divya@Divyasree:~/pipeline_performance$ ls -l build/
total 28
-rwxr-xr-x 1 divya divya 28456 Sep 28 05:49 pipeline_monitor
divya@Divyasree:~/pipeline_performance$ make run
./build/pipeline_monitor
=====
PIPELINE PERFORMANCE MONITOR
=====

Launching 4 jobs...

Job 1 started. PID = 1651
Job 1 completed successfully.
Job 2 started. PID = 1652
Job 2 completed successfully.
Job 3 started. PID = 1653
Job 3 completed successfully.
Job 4 started. PID = 1654

=====
PERFORMANCE ANALYSIS
=====

Job 1
Execution Time : 0.0082 seconds
CPU Time      : 2.20 ms
Status        : SUCCESS

Job 2
Execution Time : 0.0106 seconds
CPU Time      : 13.48 ms
Status        : SUCCESS

Job 3
Execution Time : 0.0150 seconds
CPU Time      : 29.36 ms
Status        : SUCCESS

Job 4
Execution Time : 0.0087 seconds
CPU Time      : 38.53 ms
Status        : FAILURE

=====
STABILITY
=====
Total Jobs      : 4
Successful Jobs : 3
Failed Jobs     : 1
Stability       : 75.00%

```

```

=====
OVERALL PERFORMANCE
=====
Total Execution Time : 0.0426 seconds
Average Job Time    : 0.0107 seconds
Pipeline Status     : FAILURES DETECTED

Performance analysis completed.
divya@Divyasree:~/pipeline_performance$ make clean
rm -rf build
divya@Divyasree:~/pipeline_performance$ ls
Makefile  README.md  docs  pipeline_monitor  reports  src
divya@Divyasree:~/pipeline_performance$ find . -maxdepth 2 -type f | sort
./.gitignore
./Makefile
./README.md
./docs/architecture.md
./docs/design_decisions.txt
./docs/documentation_review.txt
./docs/findings.txt
./docs/flow_diagram.txt
./docs/syscalls.txt
./pipeline_monitor
./reports/performance_report.txt
./src/pipeline_monitor.c
divya@Divyasree:~/pipeline_performance$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:     git config --global init.defaultBranch <name>
hint:
hint:     git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:     git branch -m <name>
Initialized empty Git repository in /home/divya/pipeline_performance/.git/
divya@Divyasree:~/pipeline_performance$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        Makefile
        README.md
        docs/
        reports/
        src/

nothing added to commit but untracked files present (use "git add" to track)
divya@Divyasree:~/pipeline_performance$ git config --global user.name "Your Name"
divya@Divyasree:~/pipeline_performance$ git config --global user.email "your-email@example.com"
divya@Divyasree:~/pipeline_performance$ git config --global --list
user.name=Your Name
user.email=your-email@example.com
divya@Divyasree:~/pipeline_performance$ git add .
divya@Divyasree:~/pipeline_performance$ git status
On branch master

No commits yet

```

```

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   .gitignore
    new file:   Makefile
    new file:   README.md
    new file:   docs/architecture.md
    new file:   docs/design_decisions.txt
    new file:   docs/documentation_review.txt
    new file:   docs/findings.txt
    new file:   docs/flow_diagram.txt
    new file:   docs/syscalls.txt
    new file:   reports/performance_report.txt
    new file:   src/pipeline_monitor.c

divya@Divyasree:~/pipeline_performance$ git commit -m "Organize project and add documentation"
[master (root-commit) cde1bb9] Organize project and add documentation
11 files changed, 1231 insertions(+)
create mode 100644 .gitignore
create mode 100644 Makefile
create mode 100644 README.md
create mode 100644 docs/architecture.md
create mode 100644 docs/design_decisions.txt
create mode 100644 docs/documentation_review.txt
create mode 100644 docs/findings.txt
create mode 100644 docs/flow_diagram.txt
create mode 100644 docs/syscalls.txt
create mode 100644 reports/performance_report.txt
create mode 100644 src/pipeline_monitor.c
divya@Divyasree:~/pipeline_performance$ git log --oneline
cde1bb9 (HEAD -> master) Organize project and add documentation
divya@Divyasree:~/pipeline_performance$ git status
On branch master
nothing to commit, working tree clean

create mode 100644 src/pipeline_monitor.c
divya@Divyasree:~/pipeline_performance$ git log --oneline
cde1bb9 (HEAD -> master) Organize project and add documentation
divya@Divyasree:~/pipeline_performance$ git status
On branch master
nothing to commit, working tree clean
divya@Divyasree:~/pipeline_performance$ git tag
divya@Divyasree:~/pipeline_performance$ git log --oneline --decorate -5
cde1bb9 (HEAD -> master) Organize project and add documentation
divya@Divyasree:~/pipeline_performance$ |

```

TO Create Demo Scenarios, Validate Features, Rehearse Presentation,
Demonstrate Pipelines, Demonstrate Signals, Final Project Review

```

divya@Divyasree:~$ mkdir final_demo
divya@Divyasree:~$ cd final_demo
divya@Divyasree:~/final_demo$ pwd
/home/divya/final_demo
divya@Divyasree:~/final_demo$ echo "Linux File I/O Management System" > file1.txt
divya@Divyasree:~/final_demo$ echo "Producer Consumer Deadlock Signals" > file2.txt
divya@Divyasree:~/final_demo$ echo "Operating Systems Project" > file3.txt
divya@Divyasree:~/final_demo$ ls -l
total 12
-rw-r--r-- 1 divya divya 33 Oct  1 14:35 file1.txt
-rw-r--r-- 1 divya divya 35 Oct  1 14:35 file2.txt
-rw-r--r-- 1 divya divya 26 Oct  1 14:35 file3.txt
divya@Divyasree:~/final_demo$ cat file1.txt
Linux File I/O Management System
divya@Divyasree:~/final_demo$ stat file1.txt
  File: file1.txt
  Size: 33          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 43714        Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/   divya)   Gid: ( 1000/   divya)
Access: 2026-10-01 14:35:24.575048178 +0000
Modify: 2026-10-01 14:35:07.031754819 +0000
Change: 2026-10-01 14:35:07.031754819 +0000
 Birth: 2026-10-01 14:35:07.031754819 +0000
divya@Divyasree:~/final_demo$ ls -li file1.txt
43714 file1.txt
divya@Divyasree:~/final_demo$ cp file1.txt backup.txt
divya@Divyasree:~/final_demo$ ls -l
total 16
-rw-r--r-- 1 divya divya 33 Oct  1 14:35 backup.txt
-rw-r--r-- 1 divya divya 33 Oct  1 14:35 file1.txt
-rw-r--r-- 1 divya divya 35 Oct  1 14:35 file2.txt
-rw-r--r-- 1 divya divya 26 Oct  1 14:35 file3.txt
divya@Divyasree:~/final_demo$ cat backup.txt

```

```

total 16
-rw-r--r-- 1 divya divya 33 Oct  1 14:35 backup.txt
-rw-r--r-- 1 divya divya 33 Oct  1 14:35 file1.txt
-rw-r--r-- 1 divya divya 35 Oct  1 14:35 file2.txt
-rw-r--r-- 1 divya divya 26 Oct  1 14:35 file3.txt
divya@Divyasree:~/final_demo$ cat backup.txt
Linux File I/O Management System
divya@Divyasree:~/final_demo$ mv backup.txt renamed.txt
divya@Divyasree:~/final_demo$ ls
file1.txt file2.txt file3.txt renamed.txt
divya@Divyasree:~/final_demo$ ls
file1.txt file2.txt file3.txt renamed.txt
divya@Divyasree:~/final_demo$ rm renamed.txt
divya@Divyasree:~/final_demo$ ls
file1.txt file2.txt file3.txt
divya@Divyasree:~/final_demo$ echo "Testing file descriptor" > test.txt
divya@Divyasree:~/final_demo$ cat test.txt
Testing file descriptor
divya@Divyasree:~/final_demo$ echo "Written using FD 3" >&3
-bash: 3: Bad file descriptor
divya@Divyasree:~/final_demo$ cat descriptor.txt
cat: descriptor.txt: No such file or directory
divya@Divyasree:~/final_demo$ exec 3>descriptor.txt
divya@Divyasree:~/final_demo$ echo "Written using FD 3" >&3
divya@Divyasree:~/final_demo$ exec 3>&-
divya@Divyasree:~/final_demo$ cat descriptor.txt
Written using FD 3
divya@Divyasree:~/final_demo$ ls -l descriptor.txt
-rw-r--r-- 1 divya divya 19 Oct  1 14:37 descriptor.txt
divya@Divyasree:~/final_demo$ |

```